

Dynamic Workflows com AI Agents e Sub-Agents: Padrões, Frameworks e Aplicações em Produção

OpenClaw Agency

18 de julho de 2026

Taís Lima Universidade Estadual de Santa Cruz - UESC Ilhéus, Bahia, Brasil *Autor correspondente: tais.lima@uesc.br*

Resumo

Sistemas multi-agente evoluíram de experimentos de laboratório para produção empresarial em escala, registrando um crescimento substancial no interesse corporativo e na pesquisa acadêmica. Este artigo apresenta uma síntese crítica e uma análise arquitetônica dos padrões fundamentais de orquestração de agentes de inteligência artificial (AI Agents) e sub-agentes (Sub-Agents) em ambientes de produção. São discutidos detalhadamente os padrões Orchestrator-Worker (Supervisor), Sequential Pipeline, Parallel Fan-Out/Fan-In, Dynamic Handoff (Routing), Group Chat e Magentic (Planning-Ledger). Adicionalmente, analisamos a maturidade, os diferenciais e a adequação dos principais frameworks do ecossistema, tais como LangGraph, CrewAI, OpenAI Agents SDK, Claude Agent SDK, Microsoft Agent Framework e Spring AI Agent Utils, fornecendo uma matriz de compatibilidade estruturada. O artigo conclui com diretrizes práticas de engenharia para adoção em produção, abrangendo estratégias de gerenciamento de contexto, otimização de custos de tokens, segurança orientada ao princípio do menor privilégio e observabilidade analítica.

Palavras-chave: Agentes de IA; Workflows Dinâmicos; Orquestração Multi-agente; Padrões de Arquitetura; LangGraph; CrewAI.

Abstract

Multi-agent systems have evolved from laboratory experiments to enterprise-scale production, registering substantial growth in corporate interest and academic research. This article presents a critical synthesis and architectural analysis of the fundamental orchestration patterns for artificial intelligence agents (AI Agents) and sub-agents (Sub-Agents) in production environments. We discuss in detail the Orchestrator-Worker (Supervisor), Sequential Pipeline, Parallel Fan-Out/Fan-In, Dynamic Handoff (Routing), Group Chat, and Magentic (Planning-Ledger) patterns. Furthermore, we analyze the maturity, differentials, and suitability of the main frameworks in the ecosystem, such as LangGraph, CrewAI, OpenAI Agents SDK, Claude Agent SDK, Microsoft Agent Framework, and Spring AI Agent Utils, providing a structured compatibility matrix. The article concludes with practical engineering guidelines for production adoption, covering context management strategies, token cost optimization, security oriented to the principle of least privilege, and analytical observability.

Keywords: AI Agents; Dynamic Workflows; Multi-agent Orchestration; Architectural Patterns; LangGraph; CrewAI.

1 Introdução: Por que Agents e Sub-Agents?

A inteligência artificial generativa atingiu um patamar de maturidade em que modelos de linguagem de grande porte (LLMs) resolvem tarefas isoladas com alta competência. No entanto, problemas complexos do mundo real raramente são formados por etapas estanques. Um processo empresarial típico envolve pesquisa, análise, redação, revisão, aprovação e publicação — cada estágio demandando habilidades, fontes de dados e critérios de qualidade distintos.

É nesse contexto que emergem os sistemas multi-agente. Em vez de um único modelo tentar resolver um problema complexo em uma única chamada, múltiplos agentes especializados colaboram, cada um responsável por uma parte delimitada do processo (LI et al., 2024). Cada agente pode ser configurado com seu próprio modelo de linguagem, conjunto de ferramentas, instruções de sistema e base de conhecimento. O resultado prático é um sistema mais modular, auditável e resiliente.

O valor fundamental dos sub-agentes não reside unicamente no paralelismo computacional, mas na compressão de contexto. Em sistemas mono-agente, todo o histórico da conversa e todo o conhecimento de domínio precisam ser incluídos em uma única janela de contexto. À medida que a tarefa se alonga, o contexto cresce exponencialmente, a qualidade da resposta degrada e os custos operacionais aumentam. Sub-agentes bem projetados mantêm o contexto do agente pai enxuto, reduzindo o consumo de tokens em mais de 90% e prevenindo a perda de informação em janelas de contexto saturadas (RICKI, 2026).

A questão enfrentada pelas organizações de tecnologia em 2026 não é mais se devem adotar arquiteturas multi-agente, mas qual padrão de orquestração e qual framework melhor se adaptam a seus requisitos de latência, segurança e custo.

2 O que São Dynamic Workflows?

Dynamic Workflows são arquiteturas de software nas quais múltiplos agentes de IA colaboram de forma coordenada para executar tarefas complexas. Diferentemente de pipelines de execução estáticos e rígidos, os workflows dinâmicos permitem que a topologia de execução — quais agentes são chamados, em que ordem, com que frequência — seja definida em tempo de execução com base no contexto da requisição e nos resultados parciais (YUE et al., 2026).

O conceito de workflow dinâmico apoia-se em três pilares:

Decomposição: uma tarefa complexa é dividida em subtarefas menores e mais específicas. **Delegação:** cada subtarefa é atribuída ao agente ou sub-agente mais qualificado para executá-la com base em suas especializações. **Coordenação:** os resultados parciais são agregados, refinados e integrados para produzir o resultado final.

A orquestração pode ser classificada como estática (a topologia é definida em tempo de design) ou dinâmica (a topologia emerge durante a execução por meio de decisões tomadas por LLMs). A maioria dos sistemas de produção de nível empresarial adota uma abordagem híbrida: a estrutura geral do fluxo é predefinida para garantir consistência e conformidade, mas as decisões de roteamento, replanejamento e tratamento de exceções são tomadas dinamicamente pelos próprios agentes.

3 Padrões de Orquestração em Produção

O ecossistema contemporâneo de engenharia de software voltado à inteligência artificial converge para seis padrões fundamentais de orquestração de agentes.

3.1 Orchestrator-Worker (Supervisor)

O padrão Orchestrator-Worker, frequentemente denominado Supervisor, é a arquitetura mais difundida em sistemas multi-agente comerciais. Um agente orquestrador central recebe a requisição

do usuário, decompõe o problema em subtarefas, delega cada subtarefa a um agente especialista (worker), monitora o progresso de execução e sintetiza o resultado final (MICROSOFT, 2026).

Este padrão é indicado para workflows complexos onde a decomposição da tarefa requer julgamento contextual e onde os planos de execução variam muito conforme a entrada do usuário. Por outro lado, o padrão adiciona de 15% a 30% de latência operacional e consome de 20% a 40% a mais de tokens devido às múltiplas chamadas de ida e volta entre o orquestrador e os agentes subordinados.

O Claude Agent SDK da Anthropic implementa este padrão como primitiva fundamental através da capacidade de expor agentes como ferramentas programáticas utilizando ‘agent.asTool()’. A Microsoft Agent Framework adota a mesma estrutura mapeando os agentes em grafos de fluxo de trabalho síncronos e assíncronos.

3.2 Sequential Pipeline (Prompt Chaining)

No padrão Sequential Pipeline, os agentes são encadeados em uma ordem linear estrita e predefinida (AWS, 2025). A saída gerada por um agente serve como entrada direta para o agente subsequente, estabelecendo um fluxo determinístico e sequencial.

Recomenda-se o uso deste padrão em processos com dependências lineares claras e etapas bem delimitadas, tais como ciclos de produção de conteúdo técnico (geração de rascunho → revisão gramatical → verificação de fatos → formatação). Deve ser evitado em workflows que requeiram iterações complexas ou caminhos de retorno (backtracking).

3.3 Parallel Fan-Out / Fan-In

O padrão Parallel Fan-Out/Fan-In envolve a execução simultânea de múltiplos agentes para processar a mesma tarefa a partir de diferentes especialidades. Os resultados intermediários são consolidados em um nó de agregação (Fan-In) que utiliza votação, média ponderada ou sumarização algorítmica para gerar a saída consolidada.

Este padrão é ideal para análises multi-perspectivas e tomada de decisão crítica sob restrição de tempo. Um exemplo de produção é a análise de investimentos financeiros, onde agentes executam em paralelo análises fundamentalistas, técnicas, de sentimento de mercado e ESG (ZYLOS RESEARCH, 2026), convergindo para um relatório consolidado.

3.4 Dynamic Handoff (Routing)

No padrão Dynamic Handoff, os agentes decidem autonomamente, durante a execução, quando transferir o controle da conversação para outro agente especialista (OPENAI, 2026). Diferentemente do padrão Supervisor, onde o controle sempre retorna ao orquestrador central, no Handoff a transferência de controle é definitiva ou semi-definitiva, operando um agente por vez.

Aplica-se com sucesso em centrais de atendimento ao cliente, onde um agente de triagem (router) inicial analisa as intenções do usuário e realiza a transferência para agentes especializados em suporte técnico, faturamento ou retenção. O OpenAI Agents SDK trata esta estrutura como cidadã de primeira classe por meio do método ‘handoff()’.

3.5 Group Chat (Roundtable)

O padrão Group Chat organiza múltiplos agentes em um ambiente de comunicação compartilhado (thread de chat). Um agente gerenciador (__chat manager__) ou uma lógica de roteamento em grafo define qual agente deve falar a cada turno, permitindo debate e consenso dinâmico.

É comumente aplicado em dinâmicas de __design thinking__, resolução colaborativa de problemas ou cenários com múltiplos especialistas de negócios que possuem dependências cruzadas (ex: planejamento urbano envolvendo engenheiros, planejadores ambientais e analistas de orçamento).

3.6 Magentic (Planning-Ledger)

O padrão Magentic é indicado para problemas de escopo aberto e sem fluxo de execução predefinido. O agente principal cria e atualiza dinamicamente um registro de tarefas pendentes e concluídas (`_ledger_`). Ele avalia o andamento das tarefas em tempo real, gerando novos planos de ação e acionando sub-agentes conforme o diagnóstico situacional evolui.

Este padrão é amplamente adotado em sistemas de Engenharia de Confiabilidade de Sites (SRE) e resposta a incidentes de infraestrutura tecnológica, onde o agente de resposta analisa logs, delega diagnósticos e aplica correções à medida que o comportamento do sistema é revelado (ZYLOS RESEARCH, 2026).

4 Análise de Sub-Agents e Compressão de Contexto

Um dos principais insights práticos na operação de sub-agentes em produção foi documentado por Dan Farrelly da Inngest (RICKI, 2026). Farrelly propõe a classificação das interações entre agentes e sub-agentes em três categorias operacionais distintas:

Synchronous "Wait and See": O agente pai suspende sua execução e aguarda o retorno do sub-agente. Funciona de forma análoga a uma chamada de função complexa. **Asynchronous "Fire and Forget":** O agente pai delega uma tarefa e continua seu fluxo de execução em paralelo. Útil para tarefas de processamento assíncrono ou disparos de eventos. **Scheduled "Future Execution":** O sub-agente é programado para executar em um momento futuro determinado ou sob gatilhos temporais específicos.

O valor principal desse desacoplamento não reside apenas no processamento paralelo, mas na compressão do contexto de inferência. Ao isolar sub-tarefas em sub-agentes dotados de escopos de variáveis e ferramentas restritas, reduz-se o volume total de tokens acumulados nas janelas de contexto dos modelos principais. Essa técnica previne a degradação da acurácia do modelo causada pelo fenômeno de "esquecimento no meio do contexto" (`_lost in the middle_`).

5 Ecossistema de Frameworks e Ferramentas

O mercado de software analítico para IA registrou rápida consolidação em torno de frameworks especializados em orquestração de agentes.

5.1 LangGraph

LangGraph é uma extensão do ecossistema LangChain voltada para a criação de fluxos de execução cíclicos modelados como grafos de estado. Oferece controle preciso sobre a execução e o gerenciamento de estados.

- **Diferenciais:** Suporte nativo a persistência de estados (`_checkpointing_`), depuração histórica (`_time-travel debugging_`) e integração nativa com LangSmith.
- **Limitações:** Apresenta curva de aprendizado íngreme devido à complexidade da sua API baseada em grafos e nós.

5.2 CrewAI

CrewAI adota uma metáfora organizacional baseada em equipes ("crews"), onde agentes com papéis, ferramentas e memórias predefinidas colaboram para executar tarefas hierárquicas ou sequenciais.

- **Diferenciais:** Prototipagem extremamente rápida e configuração intuitiva.
- **Limitações:** Carece de controle granular sobre o fluxo de execução para sistemas dinâmicos complexos.

5.3 OpenAI Agents SDK e Claude Agent SDK

Ambos os SDKs representam as respostas das Big Techs para a orquestração nativa de agentes. O SDK da OpenAI foca em transferências de controle de conversação (‘handoffs’) de forma simplificada, enquanto o SDK da Anthropic foca no Model Context Protocol (MCP) e na execução de agentes integrados a ambientes de desenvolvimento.

5.4 Microsoft Agent Framework e Spring AI Agent Utils

A Microsoft oferece o sucessor corporativo do AutoGen integrado ao Azure, focando em segurança e conformidade empresarial. O Spring AI Agent Utils atende ao ecossistema Java/Spring Boot, fornecendo implementações baseadas em ‘Task tools’ para isolamento de sub-agentes.

5.5 Matriz de Compatibilidade de Frameworks

A Tabela 1 apresenta a avaliação de compatibilidade nativa entre os frameworks analisados e os padrões de orquestração discutidos.

Tabela 1. Matriz de compatibilidade entre frameworks e padrões de orquestração

Framework	Orchestrator	Sequential	Parallel	Handoff	Group Chat	MCP
LangGraph	Nativo	Nativo	Nativo	Custom	Custom	Via LangChain
CrewAI	Hierárquico	Nativo	Nativo	Limitado	Não suportado	Comunitário
OpenAI Agents SDK	Nativo	Manual	Nativo	Nativo	Não suportado	Suportado
Claude Agent SDK	Nativo	Nativo	Nativo	Nativo	Nativo	Nativo
MS Agent Framework	Grafos	Grafos	Nativo	Nativo	Legado	Built-in
Spring AI Agent Utils	Nativo	Nativo	Nativo	Nativo	Não suportado	Suportado

6 Diretrizes de Engenharia para Produção

A transição de protótipos de agentes para sistemas multi-agente de produção estável exige a aplicação de rigorosas diretrizes de engenharia.

6.1 Gerenciamento Dinâmico de Contexto

À medida que múltiplos agentes interagem, a janela de contexto pode saturar rapidamente. Sistemas robustos em produção devem implementar:

- **Sumarização Incremental:** Resumo automático do histórico de mensagens antes da transferência de controle entre agentes.
- **Pruning de Histórico:** Remoção de metadados de execução irrelevantes e respostas intermediárias de ferramentas das mensagens enviadas ao modelo principal.

6.2 Otimização de Custos e Latência

A arquitetura de custos deve seguir o princípio do roteamento dinâmico baseado em complexidade (Difficulty-Aware Routing):

- **Modelos Leves:** Utilizar modelos menores e otimizados para tarefas de roteamento, extração de entidades e verificações gramaticais simples.

- **Modelos de Raciocínio:** Reservar modelos avançados de raciocínio profundo apenas para tarefas de planejamento de alto nível e consolidação analítica.

6.3 Segurança e Isolamento

Os sub-agentes devem operar sob o princípio do menor privilégio, o que significa que o acesso a APIs externas e bancos de dados deve ser restrito ao escopo exato da tarefa delegada. Adicionalmente, mecanismos de `__circuit breaker__` e cotas rígidas por agente devem ser implementados para prevenir loops de execução infinitos decorrentes de alucinações.

7 Conclusão

Os padrões de orquestração multi-agente evoluíram para se tornar pilares de arquitetura de software moderno. A escolha do padrão e framework corretos deve ser guiada por requisitos pragmáticos de latência, custos e complexidade da tarefa. A engenharia de sistemas de produção converge para o princípio de iniciar o desenvolvimento com agentes generalistas simples e aplicar a especialização em sub-agentes apenas quando houver evidências mensuráveis de ganhos de performance ou conformidade de segurança.

Referências

AHMAD, K.; MARATHE, M. Benchmark Frameworks for Multi-Agent Systems. **Journal of AI Research**, v. 45, n. 2, p. 123-145, 2026.

ANTHROPIC. **Claude Agent SDK**: Subagents in the SDK. São Francisco: Anthropic, 2026. Disponível em: <https://code.claude.com/docs/en/agent-sdk/subagents>. Acesso em: 11 jul. 2026.

AWS. **Agentic AI patterns and workflows on AWS**. Seattle: Amazon Web Services, 2025. Disponível em: <https://docs.aws.amazon.com/prescriptive-guidance/latest/agentic-ai-patterns/introduction.html>. Acesso em: 11 jul. 2026.

LI, X. et al. Survey on LLM-based Multi-Agent Systems. **Artificial Intelligence Review**, v. 38, n. 4, p. 567-590, 2024.

MICROSOFT. **AI Agent Orchestration Patterns**. Redmond: Microsoft, 2026. Disponível em: <https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/ai-agent-design-patterns>. Acesso em: 11 jul. 2026.

NIU, Y. et al. Flow: Dynamic Workflow Orchestration. In: **Proceedings of ICLR 2025**, 2025.

OPENAI. **Agents SDK**: Orchestration and handoffs. São Francisco: OpenAI, 2026. Disponível em: <https://developers.openai.com/api/docs/guides/agents/orchestration>. Acesso em: 11 jul. 2026.

RICKI. **The 3 Essential Sub-Agent Patterns for Production-Grade AI Systems**. Santa Clara: Epsilla Blog, 2026. Disponível em: <https://www.epsilla.com/blogs/2026-03-14-ai-sub-agent-patterns>. Acesso em: 11 jul. 2026.

TOMASEV, N. et al. Intelligent AI Delegation in Multi-Agent Systems. **Nature Machine Intelligence**, v. 8, n. 3, p. 234-248, 2026.

YUE, M. et al. Static to Dynamic Workflows Survey. **ACM Computing Surveys**, v. 58, n. 1, p. 1-35, 2026.

ZYLOS RESEARCH. **Agent Workflow Orchestration Patterns**: DAG, Event-Driven, and Actor Models. Boston: Zylos Research, 2026. Disponível em: <https://zylos.ai/research/2026-04-14-agent-workflow-orchestration-patterns>. Acesso em: 11 jul. 2026.